

Exploiting intra-chip locality for multi-chip GPUs via two-level shared L1 cache[☆]

Xiangrong Xu^{a,b}, Liang Wang^{a,b,*}, Limin Xiao^{a,b}, Lei Liu^{a,b}, Zihao Zhou^b, Yuanqiu Lv^{a,b}, Li Ruan^{a,b}, Xilong Xie^{a,b}, Meng Han^{a,b}, Xiaojian Liao^{a,b}

^a State Key Laboratory of Complex & Critical Software Environment, Beihang University, Beijing, Beijing, China

^b School of Computer Science and Engineering, Beihang University, Beijing, Beijing, China

ARTICLE INFO

Keywords:

Multi-chip GPUs
Locality
Cache
Memory access

ABSTRACT

Remote memory accesses in multi-chip GPUs pose a major performance bottleneck due to high latency and inter-chip bandwidth contention. Exploiting intra-chip locality alleviates this bottleneck by serving memory accesses locally and reducing cross-chip traffic. Yet, conventional coarse-grained approaches to exploiting locality in multi-chip GPUs often incur excessive overhead, limiting their potential performance benefits. To this end, we propose TLS-Cache, a two-level shared L1 cache that efficiently exploits intra-chip locality without additional cache capacity. It mitigates high-latency remote memory accesses by enabling fine-grained data reuse through cluster-shared and remote-shared L1 caches, which capture locality within and across streaming multiprocessor clusters, respectively. These two caches work cooperatively to maximize the exploitation of intra-chip locality and deliver measurable performance gains. Experimental results show that TLS-Cache improves instructions per cycle by 30.2% on average, compared with the baseline 4-chip GPU with private L1 caches.

1. Introduction

GPUs increasingly serve as the primary AI accelerators in data centers due to their generality and efficiency. With the rising costs of enhancing monolithic GPU performance through transistor scaling, multi-chip GPUs have emerged as a promising direction for future GPU architectures [1–5]. While multi-chip GPUs enable the integration of more streaming multiprocessors (SMs) and larger memory capacity, remote memory accesses, which refer to memory requests whose target data resides in memory on a different chip, pose a major performance bottleneck due to high latency and inter-chip bandwidth contention.

Fig. 1 shows a multi-chip GPU consisting of four chips, each containing multiple SMs with private L1 caches. The SMs are connected to the memory-side last-level cache and DRAM partitions via NoC. The DRAM partitions on all chips cooperatively provide a shared memory address space for the entire GPU [1]. However, the traditional cache hierarchy is inefficient in such a system because every L1 cache miss results in a request to the memory-side last-level cache. In multi-chip GPUs, accessing remote last-level caches and memory involves cross-chip traffic, which incurs additional latency. Furthermore, bandwidth

limitations and resource contention exacerbate these delays, making remote memory access a critical performance bottleneck for multi-chip GPUs.

Previous studies have introduced the remote data cache within the chip to store data from remote memory, thereby exploiting intra-chip locality, i.e., the reuse of data on the same chip, to reduce high-latency remote memory accesses [1,4,6]. The remote data cache is located between the SM's private L1 cache and the last-level cache, shared by all SMs on the local chip, and is dedicated to caching remote data from other chips. It handles L1 cache miss requests, preventing unnecessary high-latency remote memory accesses. However, to avoid additional transistor overheads, existing designs often introduce the remote data cache by reducing the capacity of other expensive on-chip memory, such as the last-level cache. This trade-off can increase the last-level cache miss rate and introduce unnecessary bandwidth resource contention, thereby reducing the overall benefits of locality exploitation [7,8]. Furthermore, while a large portion of the intra-chip locality can be exploited directly within the L1 cache in multi-chip GPUs [9–13], the remote data cache architecture requires all remote

[☆] This work was supported in part by the National Key R&D Program of China under Grant No. 2023YFB4503100; in part by the National Natural Science Foundation of China under Grants 62272026 and 62104014; in part by the Fundamental Research Funds for the Central Universities, China under Grant No. YWF-23-Q-1015; in part by State Key Laboratory of Complex & Critical Software Environment under Grant No. CCSE-2024ZX-10.

* Correspondence to: 37 Xueyuan Road, Haidian District, Beijing, 100191, P.R. China.

E-mail addresses: xrx0930@buaa.edu.cn (X. Xu), lwang20@buaa.edu.cn (L. Wang), xiaolm@buaa.edu.cn (L. Xiao), liulei2010@buaa.edu.cn (L. Liu).

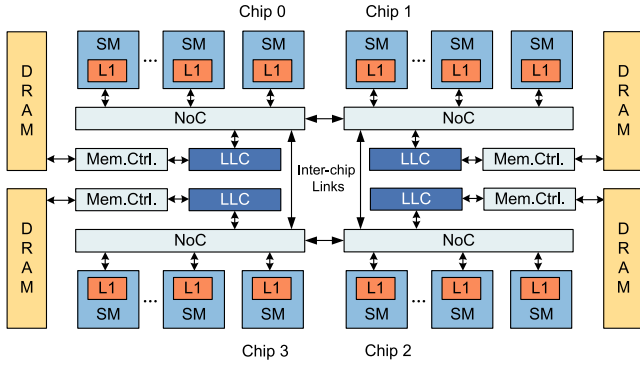


Fig. 1. Multi-chip GPU architecture consisting of four GPU chips.

memory requests to go through the remote data cache to exploit such locality. Therefore, although coarse-grained locality exploitation through the remote data cache is simple and easy to implement, the high overhead involved in leveraging intra-chip locality significantly limits the potential for performance improvement.

We observe that in multi-chip GPUs, the capacity of the L1 cache can be utilized to design a unified architecture for the L1 and remote data cache. We propose TLS-Cache, a novel two-level shared L1 cache for multi-chip GPUs that mitigates high-latency remote memory accesses. We first group the SMs into clusters and design a shared L1 cache architecture that connects the private L1 caches of multiple SMs via a two-level crossbar to enable data sharing. We then organize the shared L1 caches into two levels: the **cluster-shared L1 cache** and the **remote-shared L1 cache**. The cluster-shared L1 cache is accessed by all requests from the SMs within a cluster, enabling data sharing to exploit intra-cluster locality, i.e., data reuse among SMs within a single cluster. When a request misses in the cluster-shared L1 cache, it is routed based on the target address. If it targets local memory,¹ the request is directed to the local last-level cache. Otherwise, it is forwarded to the remote-shared L1 cache, specifically designed to store remote data and exploit inter-cluster locality, which refers to the reuse of data across different SM clusters. In addition, we introduce a request router that directs requests to the appropriate cluster-shared L1 cache or remote-shared L1 cache, since requests from SMs are no longer restricted to their private L1 cache. Our two-level shared L1 cache does not require a separate, additional remote data cache and enables fine-grained exploitation of both intra-cluster and inter-cluster locality, thereby minimizing access latency and improving the overall performance of multi-chip GPUs.

In summary, we make the following contributions:

- We observe that the shared L1 cache provides an opportunity to design a unified architecture for L1 and remote data cache, which exploits intra-chip locality without the need for additional cache capacity.
- We propose TLS-Cache, a novel two-level GPU shared L1 cache architecture that enables fine-grained intra-chip data reuse, reducing high-latency remote memory accesses and improving multi-chip GPU performance.
- Our experiments demonstrate that TLS-Cache improves performance by 30.2% compared with the baseline multi-chip GPU. It also outperforms the remote data cache and clustered shared L1 cache by 8.8% and 19.6%, respectively.

¹ In this paper, local memory refers specifically to the DRAM on the local chip, which is distinct from the concept of local memory in the CUDA programming model.

2. Background and motivation

2.1. Baseline multi-chip GPU memory system

The memory system of a multi-chip GPU follows a typical Non-Uniform Memory Access (NUMA) architecture [3,4,14,15], consisting of the memory systems of multiple chips. Each chip's memory system includes the private L1 caches within SMs and a memory subsystem that comprises the last-level cache and DRAM. The memory subsystem in each chip is accessible to SMs across all chips [8].

When an application runs on a multi-chip GPU, thread blocks are assigned to the SMs on each chip. During the execution of memory instructions, the Load/Store (LD/ST) units within the SMs generate memory access requests [16,17]. These requests are first directed to the L1 cache of the SM. When a request misses in the L1 cache, it is forwarded to the last-level cache. The request to the local memory is sent to the local last-level cache, while the request to the remote memory is transferred to another chip via the inter-chip links to access the remote last-level cache.

Memory access requests to remote chips generally experience substantially higher latency than those to local chips. This is primarily due to the time required for inter-chip data transfer, as well as additional delay caused by bandwidth limitations and resource contention [1]. As a result, despite the numerous memory optimizations proposed in the prior studies [18–23], remote memory access remains a critical performance bottleneck in multi-chip GPUs [24,25]. Exploiting data locality to minimize remote memory accesses is a promising approach to improving multi-chip GPU performance.

2.2. Remote data cache

The remote data cache architecture is widely used in multi-chip GPUs to mitigate the high latency of remote memory accesses by exploiting intra-chip locality [1,4,5,25]. It adds an extra cache level between the L1 and last-level caches on each chip, accessible by all SMs via the on-chip interconnect. Unlike conventional cache hierarchies, the remote data cache is dedicated solely to storing remote data, specifically data fetched from memory residing on other chips. When an L1 cache miss request requires remote memory access, the request is forwarded to the remote data cache. Upon a cache hit, the requested data is returned directly to the requesting SM, avoiding cross-chip traffic.

Although the remote data cache exploits intra-chip locality and reduces remote memory accesses, its introduction comes with significant overhead. First, the remote data cache requires additional cache capacity. To minimize transistor overhead, the introduction of the remote data cache typically results in reducing the capacity of other expensive on-chip memory resources, such as the last-level cache. Such reductions lead to performance degradation. Second, the remote data cache exploits intra-chip locality in a coarse-grained manner. In multi-chip GPUs, a large portion of intra-chip locality can be exploited directly at the L1 cache level [10–13,26]. However, the remote data cache architecture overlooks this. As a result, all memory requests must pass through the remote data cache to exploit such locality. To illustrate the impact of coarse-grained locality exploitation, we compare the L1 cache hit rate and remote data cache access rate (i.e., the ratio of remote data cache accesses to total memory requests) under coarse-grained and fine-grained configurations. As shown in Fig. 2, fine-grained locality exploitation increases the L1 cache hit rate from 22.4% to 32.3% and reduces the remote data cache access rate from 59.3% to 52.1%. Therefore, although the coarse-grained approach is simple and easy to implement, it leads to unnecessary requests to the remote data cache, increasing resource contention and access latency, and ultimately diminishing the performance benefits.

In summary, the remote data cache provides a simple approach to exploiting intra-chip locality and reducing remote memory accesses. However, it incurs extra cache overhead that leads to performance trade-offs and fails to capture fine-grained reuse opportunities due to its coarse-grained design.

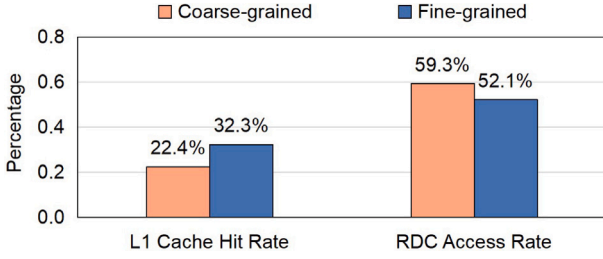


Fig. 2. L1 cache hit rate and remote data cache access rate under coarse-grained and fine-grained locality exploitation.

2.3. Shared L1 cache in monolithic GPU

In a typical two-level cache architecture of a monolithic GPU, each SM has a private L1 cache, and all SMs share a banked last-level cache. When the same cache line is requested by multiple SMs, intra-chip locality across SMs arises. However, the private L1 cache cannot exploit this locality, resulting in inefficiency.

The shared L1 cache architecture interconnects SMs through a NoC, allowing multiple SMs to share their private L1 caches. This design efficiently exploits intra-chip locality across SMs through cache sharing, thereby improving the L1 cache hit rate and reducing accesses to the last-level cache [10–13,26]. As the number of SMs in modern GPUs increases, existing shared L1 cache architectures typically organize SMs into multiple clusters, where only SMs within the same cluster share their L1 caches. This clustered approach aims to ensure that the performance benefits of cache sharing outweigh the resource contention overhead.

However, existing shared L1 cache architectures are not well-suited for multi-chip GPU systems. First, the high cost of inter-chip memory accesses in multi-chip GPUs necessitates a greater focus on intra-chip locality exploitation, which demands more advanced design strategies. While clustered shared L1 cache improves the performance of monolithic GPUs by exploiting intra-cluster locality, this approach is inefficient in multi-chip GPUs as it overlooks inter-cluster locality, resulting in unnecessary remote memory accesses. Second, conventional shared L1 cache architectures are not specifically designed to handle remote memory data, even though remote memory accesses experience much higher latency and therefore benefit more from caching compared with local memory accesses.

Overall, the shared L1 cache architecture efficiently exploits locality in monolithic GPUs, but it performs poorly when directly applied to multi-chip GPU systems.

2.4. Design considerations

We observe that both the L1 cache and remote data cache in multi-chip GPUs aim to exploit intra-chip locality, which creates an opportunity to design a unified architecture for both caches. In other words, part of the L1 cache can be dedicated to caching remote data, enabling remote data caching without increasing the overall cache capacity. We consider the following two questions.

(1) *How to exploit intra-chip locality with private L1 cache?* The GPU's L1 cache is private to each SM, allowing it to exploit locality between different thread blocks executing on a single SM. However, it cannot exploit the locality between different SMs within a chip. More importantly, the remote data cache is shared among all SMs within a chip, which means the private L1 cache cannot be directly reconfigured as a remote data cache.

(2) *How to mitigate the performance degradation caused by the reduction of L1 cache capacity?* Without increasing the cache capacity, dedicating part of the L1 cache to caching remote data reduces the

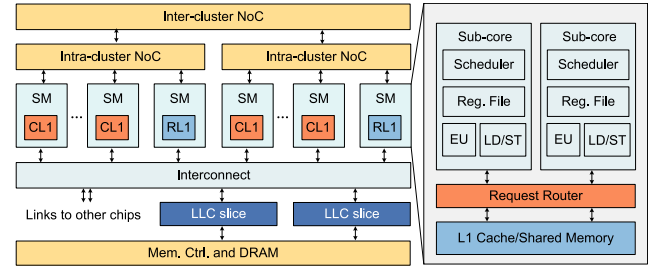


Fig. 3. Overview of TLS-Cache. CL1 represents the cluster-shared L1 cache, and RL1 represents the remote-shared L1 cache.

overall available L1 cache capacity, leading to more cache misses and resulting in performance degradation.

Fortunately, the shared L1 cache architecture efficiently addresses these two questions. In this architecture, each L1 cache is accessible by multiple SMs to exploit intra-chip locality at the L1 cache level and can be easily reconfigured as a remote data cache. Additionally, compared with the private L1 cache, the shared L1 cache reduces data duplication, thus increasing the available cache capacity. Therefore, even if part of the cache capacity is dedicated to caching remote memory data, the performance degradation is mitigated. This insight enables the design of a novel two-level shared L1 cache architecture that integrates the L1 cache and the remote data cache into a unified design. This architecture enables fine-grained utilization of intra-chip locality without requiring additional cache capacity.

3. Design of TLS-Cache

This section presents the design of TLS-Cache, a two-level shared L1 cache architecture that exploits data locality within the chip to reduce remote memory accesses, thereby improving the overall performance of multi-chip GPUs.

3.1. Overview

Fig. 3 provides a high-level overview of TLS-Cache. For simplicity, it only shows a single GPU chip and the corresponding DRAM partition in a multi-chip GPU. The architecture of the remaining GPU chips is the same as that of this chip. We design TLS-Cache by modifying the original multi-chip GPU architecture.

As shown in Fig. 3, we first design a basic shared L1 cache architecture that organizes SMs into clusters and connects them using a two-level crossbar within the chip, enabling data exchange between the SMs. We then design a unified architecture for the L1 and remote data cache, in which the L1 cache of each SM within a cluster is allocated as either a cluster-shared L1 cache or a remote-shared L1 cache. The cluster-shared L1 cache is shared among the SMs within the cluster, exploiting intra-cluster locality, while the remote-shared L1 cache stores remote memory data and is shared across all SMs on the chip, leveraging inter-cluster locality. Requests from the LD/ST units first access the cluster-shared L1 cache. If there is a miss, the request is directed to either the local memory or the remote-shared L1 cache, depending on the access address. A hit in the remote-shared L1 cache eliminates the need for high-latency remote memory accesses, alleviating the performance bottleneck. Both types of shared L1 caches utilize the existing L1 cache from the original architecture, thus avoiding the need for additional cache capacity. Furthermore, since requests from an SM are no longer restricted to accessing its private L1 cache, we introduce a request router between the LD/ST units and the L1 cache, which directs requests to the appropriate cache. We detail our design below.

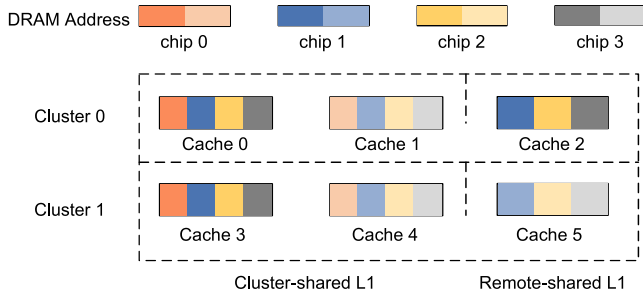


Fig. 4. Address mapping of the cluster-shared and remote-shared L1 caches.

3.2. Basic GPU shared L1 cache

We modify the traditional GPU L1 cache to develop a basic shared L1 cache architecture for multi-chip GPUs. As shown in Fig. 3, the SMs in each chip are organized into multiple clusters. To enable data exchange between SMs, we introduce a two-level crossbar. The first-level crossbar, called the intra-cluster crossbar, facilitates data sharing within the same cluster. The second-level crossbar, called the inter-cluster crossbar, allows data exchange between clusters and enables SMs to access the L1 caches in other clusters. By employing a two-level crossbar to interconnect all SMs within the chip, the introduced overhead is minimized. Compared with other interconnects, such as ring network [12,26], mesh [27], and the single-level crossbar [11] used in conventional shared L1 caches, the two-level crossbar not only supports more SMs within the chip but is also better suited for integration with our two-level shared L1 cache design.

In the basic shared L1 cache architecture, each SM can access the L1 caches of other SMs within the same cluster via the intra-cluster crossbar. It is worth noting that our shared cache design only works for global memory requests. For other types of requests, such as shared memory and texture requests, our design is consistent with the original GPU architecture.

3.3. Unified cache architecture

After enabling intra-chip sharing of L1 caches in multi-chip GPUs, we design a unified cache architecture for the cluster-shared L1 cache and the remote-shared L1 cache. In our design, each L1 cache is configured as either a cluster-shared L1 cache or a remote-shared L1 cache by modifying the address mapping policy of the original L1 cache.

Cluster-shared L1 cache. The cluster-shared L1 caches are the first-level caches that handle all requests from the SMs. Each cluster-shared L1 cache is mapped to a distinct segment of the address space, and all cluster-shared L1 caches within a cluster collectively cover the entire address space. As shown in Fig. 4, consider a multi-chip GPU consisting of four chips, each with its own DRAM. Each chip contains six SMs, organized into two clusters. Fig. 4 illustrates the L1 caches on chip 0, where each cluster is equipped with two cluster-shared L1 caches, resulting in a total of four cluster-shared L1 caches: Cache 0, Cache 1, Cache 3, and Cache 4. In the figure, different colors represent different address segments. Within the same cluster, Cache 0 and Cache 1 store data from different address segments and collectively cover the entire address space. To avoid resource contention caused by excessive sharing, there is no sharing between L1 caches in different clusters. Therefore, Cache 3 and Cache 4 together also cache the entire address space.

The cluster-shared cache avoids data duplication by mapping memory requests for the same address from different SMs to the same cache, thereby increasing the available L1 cache capacity. Therefore, although some cache capacity is reserved for the remote-shared L1 cache in

each cluster, the performance of the cluster-shared L1 cache does not degrade significantly.

Compared with the remote data cache, the cluster-shared cache efficiently exploits intra-chip locality at the L1 cache level, thereby reducing the latency for exploiting locality.

Remote-shared L1 cache. The remote-shared L1 cache is located between the cluster-shared L1 cache and the last-level cache, exclusively caching data from remote memory. When a memory request misses in the cluster-shared L1 cache and needs to access remote memory, it first accesses the remote-shared L1 cache.

The remote-shared L1 cache is also implemented by modifying the address mapping strategy of the original L1 cache. Each remote-shared cache is mapped to a specific segment of the remote memory address space. The remote-shared L1 caches of all clusters on the chip collectively cache the entire remote memory address space. As shown in Fig. 4, Cache 2 and Cache 5 serve as remote-shared L1 caches. They are located on chip 0 and are responsible for caching data from the DRAMs of chips 1–3. Each cache handles a distinct portion of the address space, and together, they cover the entire remote memory address space.

Unlike the cluster-shared L1 cache, which is accessible only to SMs within the cluster, the remote-shared L1 cache is accessible to all SMs on the chip. The remote-shared L1 cache exploits locality between different clusters. Therefore, requests that hit the remote-shared L1 cache within the chip can avoid high-latency remote memory accesses.

The proposed two-level shared L1 cache architecture exploits both intra-cluster and inter-cluster data locality at fine granularity to improve the performance of multi-chip GPUs. In this design, each L1 cache can be configured as either a cluster-shared L1 cache or a remote-shared L1 cache, depending on the overall system configuration. It is worth noting that, in terms of cache coherence mechanisms, each cache line within the cluster-shared L1 caches is maintained as a single copy across all L1 caches in a cluster. This eliminates intra-cluster data duplication and avoids the need for additional cache coherence mechanisms within the cluster. Although data copy can exist between clusters, this is consistent with the data duplication inherent in traditional GPU private L1 caches, allowing the current default cache coherence mechanisms in GPUs to manage it efficiently. Similarly, for the remote-shared L1 cache, the default cache coherence mechanisms in GPUs are also sufficient.

3.4. Request router

In the shared cache design, requests from an SM are no longer restricted to accessing the private L1 cache of the SM, but can also access the L1 caches of other SMs. To achieve this, we introduce a request router that directs requests to the appropriate L1 cache. The request router is located between the LD/ST units and the L1 cache within the SM. It takes memory access requests from the LD/ST units or the cluster-shared L1 caches as inputs and selects the target cache based on the address tag of each request.

Fig. 5 illustrates the hardware organization of the request router. The request router consists of two modules, one of which handles memory requests from the LD/ST units, and the other handles requests from the cluster-shared L1 cache. When a request from the LD/ST unit arrives at the request router (Q1), the router directs the request to the appropriate cluster-shared L1 cache. The request first passes through the Tag Selector, which selects the specific bits of the address tag to determine the target cache. In our design, each cluster contains N cluster-shared L1 caches. We use $\lceil \log_2 N \rceil$ bits in the address tags to map a request to the corresponding cluster-shared L1 cache. The request is then forwarded to the Cache Mapping Unit, which maps the request to a specific cluster-shared L1 cache based on mapping rules described in Section 3.3. If the target cluster-shared L1 cache is in the same SM, the request is sent directly to it (Q2). Otherwise, the request is routed through the intra-cluster crossbar to access the L1 caches of the other SMs within the same cluster (Q3).

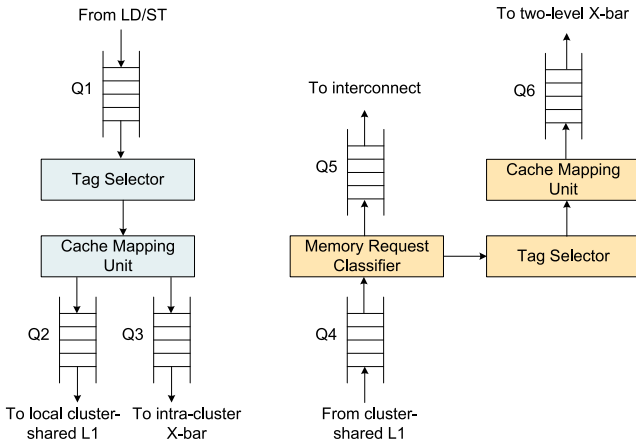


Fig. 5. Hardware organization of request router.

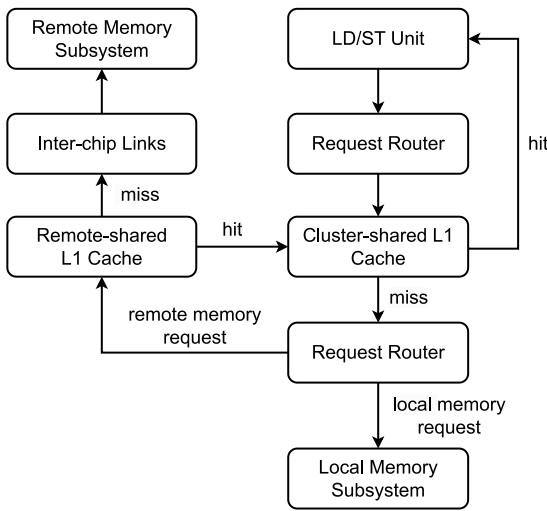


Fig. 6. Memory access flow in the TLS-Cache.

When a request originates from the cluster-shared L1 cache (Q4), it indicates a miss in the cluster-shared L1. The request is first forwarded to the Memory Request Classifier, which determines whether it should be sent to local or remote memory by locating the page containing the target address. If local memory is accessed, the request is routed to the local last-level cache through the interconnect (Q5). Otherwise, similar to accessing the cluster-shared L1 cache, the request selects specific bits of the address tag in the Tag Selector and uses the Cache Mapping Unit to determine the target cache. It then proceeds through the two-level crossbar to the remote data cache (Q6).

3.5. Working example

We provide a working example to demonstrate the access path of a request in the TLS-Cache architecture. As shown in Fig. 6, when the warp scheduler in the sub-core of an SM issues a memory instruction, the LD/ST unit generates memory access requests and sends them to the request router.

The request router determines the target L1 cache for each request based on the address mapping policy. If the target cluster-shared L1 cache is in the current SM, the request accesses the local L1 cache directly. If the target L1 cache is in another SM within the same cluster, the request bypasses the local cache and miss status handling registers (MSHR), accessing the target L1 cache through the intra-cluster crossbar.

When the request arrives at the target cluster-shared L1 cache, it checks for a cache hit by comparing the address tag with the tag array, following a process similar to that of accessing a private L1 cache. If a cache hit occurs, the request is routed directly to the LD/ST units of the requesting SM, completing the data access. If the request misses in the cluster-shared L1 cache, it is forwarded to the request router, which determines whether local or remote memory should be accessed. If local memory is to be accessed, the request bypasses the remote-shared L1 cache and proceeds directly to the last-level cache through the interconnect between the SMs and the local memory subsystem. Otherwise, the request is routed to the remote-shared L1 cache via the two-level crossbar.

When the request hits the remote-shared L1 cache, the request no longer needs to travel across the chip to access the remote memory, but fills the cluster-shared L1 cache and writes back to the original SM. If the request misses the remote-shared L1 cache, it travels across the inter-chip links to the remote memory subsystem to access the corresponding last-level cache and remote memory.

3.6. Comparison of cache architectures

We conduct a detailed comparative analysis to highlight the key advantages of our TLS-Cache over the remote data cache architecture. In Fig. 7, we compare three cache architectures: private L1 cache, remote data cache, and our proposed TLS-Cache, focusing on how to exploit locality for handling memory requests. We consider only memory requests that hit on-chip caches in multi-chip GPUs. These requests are categorized as follows:

R1: Requests that exploit intra-cluster locality within the same SM.

R2: Requests that cannot exploit intra-cluster locality within the same SM, but exploit intra-cluster locality across SMs.

R3: Requests that cannot exploit intra-cluster locality, but exploit inter-cluster locality within the same chip.

R4: Local memory requests that cannot exploit intra-chip locality, but exploit inter-chip locality within the GPU, referring to the reuse of data across chips.

R5: Remote memory requests that cannot exploit intra-chip locality, but exploit inter-chip locality within the GPU.

It is worth noting that R1 and R2 leverage intra-cluster locality, while R3 exploits inter-cluster locality. Both intra-cluster locality and inter-cluster locality are types of intra-chip locality, which is the locality we aim to exploit.

Fig. 7(a) illustrates the traditional private cache hierarchy in a multi-chip GPU, where only locality within the same SM can be exploited. As a result, while R1 hits the L1 cache, all other requests must access the last-level cache, leading to numerous remote memory accesses (e.g., R2, R3, R5). These high-latency remote accesses pose a significant performance bottleneck in multi-chip GPUs.

As shown in Fig. 7(b), under the remote data cache architecture, R1 results in a cache hit, while other requests result in cache misses. Requests targeting local memory (e.g., R4) are forwarded to the local last-level cache, whereas remote memory requests (e.g., R2, R3, R5) are directed to the remote data cache. The remote data cache leverages intra-chip locality, enabling R2 and R3 to hit, thus reducing high-latency remote memory accesses. However, R2 can complete data access by exploiting locality at the L1 cache level, which the coarse-grained design of the remote data cache architecture fails to exploit, thereby limiting potential performance gains.

In contrast, TLS-Cache exploits intra-chip locality in a fine-grained manner. As shown in Fig. 7(c), the cluster-shared L1 cache captures locality both within and across SMs in a cluster, enabling R1 and R2 to hit without accessing higher cache levels. Requests missing in the cluster-shared L1 cache are directed either to the local last-level cache for local memory access (e.g., R4) or to the remote-shared L1 cache for remote memory access (e.g., R3, R5). By exploiting inter-cluster locality, the remote-shared L1 cache reduces high-latency remote memory accesses.

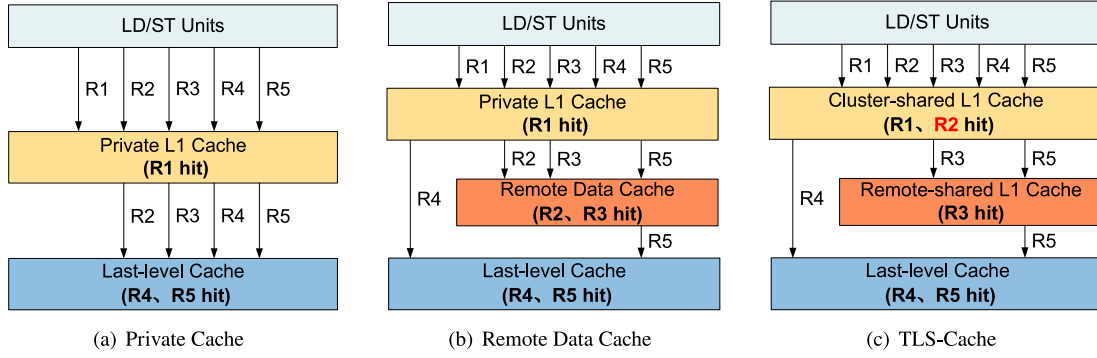


Fig. 7. Comparison of cache architectures.

Table 1
Configuration parameters of the simulated multi-chip GPU.

Parameters	Value
Number of chips	4
SM features	256 SMs, 64 SMs per chip, 1 GHz, 4 sub-cores, 4 warp schedulers
L1 cache per SM	128 KB sector L1 data cache, 4 banks, LRU, 128B cache line, 4 sectors, latency = 20 cycles
Last-level cache per chip	256 KB per slice (4 MB in total), 128B cache line, 32B sector size, latency = 160 cycles
Memory per chip	8 Memory Ctrl., 16 banks, latency = 100 cycles
Intra-chip NoC	64 × 16 crossbar, 1 GHz, 40B flit size, in/out buffer limit 512, iSLIP Arbitration type
Inter-chip links	768 GB/s, ring, 32 cycles/hop
Page allocation	4 KB Page, first-touch

In summary, compared with the remote data cache architecture, TLS-Cache exploits locality in a more fine-grained manner, thereby reducing the access distance to exploit locality and minimizing unnecessary access latency.

4. Experimental evaluation

4.1. Experimental setup

We extend Accel-Sim [17], a state-of-the-art cycle-level GPU performance simulator, to support multi-chip GPU simulation and simulate our design faithfully. Table 1 shows the detailed configuration of the baseline multi-chip GPU for our simulations, which is in line with prior work and commercial GPUs [1,7,28]. As shown in the table, our baseline multi-chip GPU consists of four chips. Each chip contains 64 SMs and 16 last-level cache slices, corresponding to eight memory channels. In total, the multi-chip GPU has 256 SMs and 32 memory channels. Within a single chip, the on-chip interconnect uses a 64x16 crossbar to connect the SMs to the memory subsystem. Multiple chips are interconnected using a ring topology, providing a bandwidth of 768 GB/s and a latency of 32 cycles per hop. We adopt the first-touch page allocation policy, which is a commonly used page allocation policy for multi-chip GPUs, which maps a page to the memory partition that first accesses a cache block within that page [1,7].

The applications studied in this work, listed in Table 2, are selected from benchmark suites commonly used to evaluate multi-chip GPU architectures [1,2,33]. From these suites, we specifically select applications that exhibit high intra-chip locality, as TLS-Cache is designed to capture and exploit intra-chip data reuse to improve performance. To experimentally obtain the value of intra-chip locality for the application, we assume an ideal multi-chip GPU, where each chip is equipped with a sufficiently large remote data cache. The rest of the

Table 2
Simulated workloads.

Benchmark	Remote data cache hit rate	Remote memory access frequency	Intra-chip locality
SS [29]	0.93	0.54	0.50
SRAD [30]	0.86	0.60	0.52
FFT [31]	0.92	0.63	0.58
SPMV [32]	0.96	0.75	0.72
SORT [31]	0.97	0.74	0.72
CFD [30]	0.97	0.75	0.72
GEMM [31]	0.98	0.75	0.74
ST2D [31]	1.00	0.74	0.74
REDC [31]	1.00	0.75	0.75

Table 3
Comparison schemes.

Abbreviation	Scheme
Baseline	Baseline multi-chip GPU
Shared L1	Multi-chip GPU with cluster-shared L1 cache
L1.5	Multi-chip GPU with remote data cache
TLS-Cache	Multi-chip GPU with two-level shared L1 cache

configurations are provided in Table 1. In this ideal multi-chip GPU, all remote memory requests that miss in the L1 cache are directed to the remote data cache, efficiently capturing almost all intra-chip data locality. We calculate the intra-chip locality using Eq. (1).

$$ICL = \frac{R_Remote_Hits}{R_Remote_Total} \times \frac{R_Remote_LLC}{R_Total_LLC} \quad (1)$$

In Eq. (1), ICL represents the value of intra-chip locality, R_Remote_Hits represents the number of remote data cache read hits. R_Remote_Total represents the total number of remote data cache read requests. We calculate locality using the number of read requests, as read instructions are often dependencies for subsequent instructions, and their latencies significantly impact application performance. R_Remote_LLC represents the total number of remote memory access requests, while R_Total_LLC represents the total number of last-level cache access requests. The ratio of these two values indicates the frequency of remote memory accesses.

Table 3 presents the architectural schemes evaluated in this work. We consider four configurations: the baseline multi-chip GPU (Baseline), the clustered shared L1 cache described in Section 2.3 (Shared L1), the multi-chip GPU that uses part of the last-level cache capacity as a remote data cache (L1.5), and the proposed TLS-Cache. To ensure fairness, the size and bandwidth of the L1 caches are equivalent across all architectures. In both Shared L1 and TLS-Cache, the SM cluster size is configured as 4. For TLS-Cache, all memory requests are first directed to the cluster-shared L1 cache. Only those requests that miss in the cluster-shared L1 cache and need to access remote memory are subsequently forwarded to the remote-shared L1

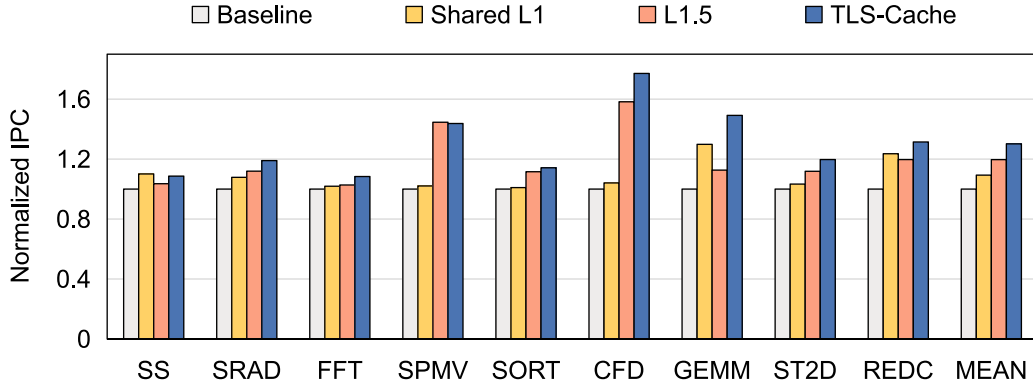


Fig. 8. Illustrating the benefits of TLS-Cache in terms of overall IPC (normalized to Baseline).

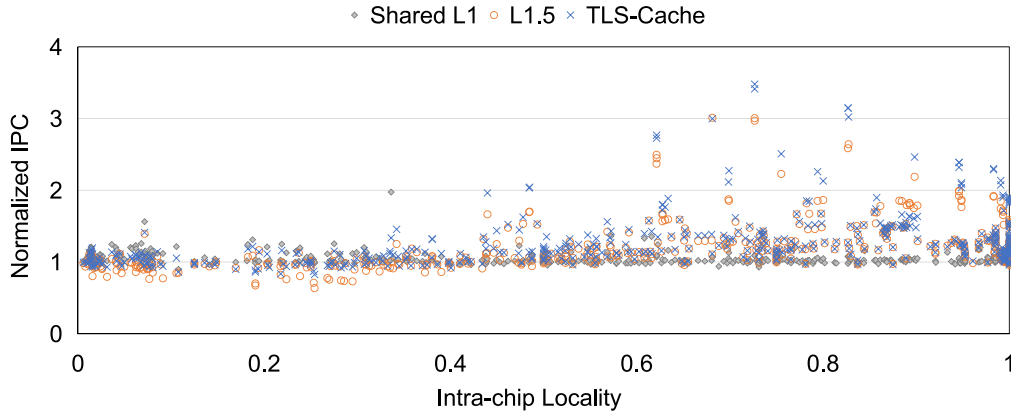


Fig. 9. Performance of kernels for GPU applications (normalized to Baseline).

cache. As a result, the capacity and bandwidth demands of the cluster-shared L1 cache are higher than those of the remote-shared L1 cache. Therefore, for a cluster size of 4, we adopt a configuration with three cluster-shared L1 caches and one remote-shared L1 cache. Additionally, the size and bandwidth of the remote data cache are the same for both TLS-Cache and L1.5. We observe the following.

4.2. Overall performance

We use IPC (instructions per cycle) to represent performance. Fig. 8 shows that for applications with good intra-chip locality, TLS-Cache achieves an average performance improvement of 30.2%, with a maximum improvement of 77.2% over Baseline. We conduct a paired samples t-test and demonstrate that TLS-Cache achieves statistically significant performance improvements over the baseline at the 95% confidence level.

To better understand where the performance improvement comes from, we perform a contribution analysis. Specifically, we analyze the performance under the Shared L1, which is functionally equivalent to the cluster-shared L1 in TLS-Cache, as both capture intra-cluster locality only. Compared to Baseline, Shared L1 improves performance by 9.3%. TLS-Cache further improves performance by 19.6% over Shared L1, indicating that the additional benefit comes from efficiently leveraging inter-cluster locality through the remote-shared L1 cache.

We also compare TLS-Cache with L1.5, which exploits intra-chip locality through the additional remote data cache. L1.5 improves performance by 19.7% over Baseline, while TLS-Cache further outperforms L1.5 by 8.8%. This improvement is because our two-level shared L1 cache exploits intra-chip locality in a fine-grained manner, resulting in higher performance gains.

Notably, the performance improvement provided by TLS-Cache does not show a strict correlation with the increase in intra-chip locality. We

infer that this is due to the varying sensitivity of different applications to high-latency memory accesses. For memory-sensitive applications, such as *CFD* and *GEMM*, TLS-Cache significantly improves performance by reducing high-latency remote memory accesses, achieving improvements of 77.2% and 49.2%, respectively.

4.3. Performance per kernel

GPU applications typically consist of multiple kernels. We analyze the performance improvements of kernels with intra-chip locality under Shared L1, L1.5, and TLS-Cache.

Fig. 9 illustrates the performance improvement of all kernels in our simulated workloads, excluding kernels with no locality. Each dot in the figure represents a kernel under a specific architecture. The x-axis shows the kernel's intra-chip locality, and the y-axis shows the performance improvement compared with Baseline under that architecture.

In Fig. 9, most of the blue dots representing TLS-Cache are positioned above the yellow dots representing L1.5 and the gray dots representing Shared L1, indicating that TLS-Cache provides greater performance improvements than L1.5 and Shared L1. For memory-sensitive kernels with high intra-chip locality, TLS-Cache achieves up to a 2x performance improvement over Baseline.

It is worth noting that for kernels with poor intra-chip locality, many of the yellow points have y-axis values below 1. This indicates that for kernels with poor locality, the performance under L1.5 is lower than that under Baseline, due to the performance degradation caused by the reduced last-level cache capacity. In contrast, TLS-Cache avoids this issue, suggesting that our unified architecture for L1 and remote data cache experiences less performance degradation compared with L1.5 for kernels with poor locality.

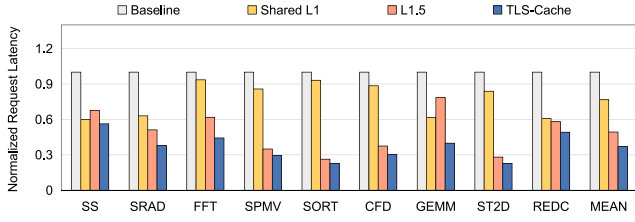


Fig. 10. Comparison of access latency for read requests across different architectures.

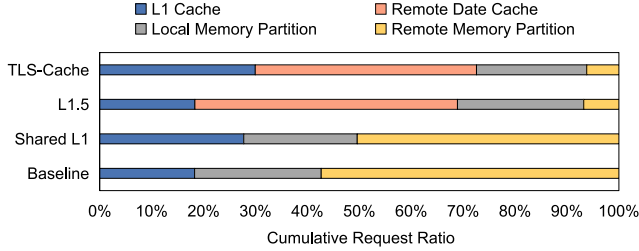


Fig. 11. Cumulative distribution of memory requests served by each memory hierarchy level (averaged across all applications).

4.4. Read request latency

We compare the read request latency to demonstrate that TLS-Cache reduces the average access latency significantly, addressing the performance bottleneck in multi-chip GPUs. We focus on the read requests because read instructions are typically dependent on subsequent instructions, thereby affecting the overall execution cycles of the application. Fig. 10 shows the read request access latency of GPU applications under different architectures. As shown in the figure, the average read memory latency of applications with high intra-chip locality is significantly reduced under the TLS-Cache. For example, in *SORT*, read latency decreases to 22.8% of that under Baseline, resulting in overall performance improvements of 14.2%, despite the application being less sensitive to memory latency. In contrast, for memory-sensitive applications such as *CFD*, the average read memory access latency decreases to 30.1% of that under Baseline, and the overall performance is improved by 77.2%. TLS-Cache efficiently mitigates numerous unnecessary high-latency memory accesses, leading to considerable performance improvements.

On average, compared with Baseline, Shared L1 reduces the average read latency to 76.7% of that under Baseline, L1.5 reduces the latency to 49.3%, and TLS-Cache reduces the latency to 36.9%. To further investigate why TLS-Cache outperforms other designs in reducing memory access latency, we analyze the breakdown of memory request destinations across different architectures. Fig. 11 presents the average cumulative ratio of requests served by each memory hierarchy level across all applications, including the L1 cache, remote data cache, local memory partition, and remote memory partition. Compared with Baseline, TLS-Cache increases the proportion of requests served at the L1 cache and additionally serves 42.6% of requests through the remote-shared L1 cache. This significantly reduces remote memory accesses, which alleviates inter-chip bandwidth contention. While Shared L1 achieves a similar L1 cache hit rate, as both leverage intra-cluster locality, TLS-Cache further captures inter-cluster locality through the remote-shared L1 cache, resulting in fewer cross-chip memory accesses. Compared with L1.5, TLS-Cache improves the L1 cache hit rate from 18.3% to 30.0% and avoids the additional latency introduced by accessing the higher-latency remote data cache. As a result, for applications with high intra-chip locality, TLS-Cache achieves lower average read latency than all three comparison architectures due to fewer high-latency accesses and reduced inter-chip bandwidth contention.

4.5. Sensitivity studies

Effect of SM Count within each chip. We vary only the number of SMs per chip in a multi-chip GPU, while maintaining all other configurations unchanged, to observe the performance of the four architectures under different configurations. The performance of each architecture is normalized to that of Baseline. The results are shown in Fig. 12(a). We observe that TLS-Cache achieves noticeable performance gains compared with Baseline in configurations with 80 SMs and 96 SMs per chip, with average improvements of 15.6% and 27.6%, respectively. However, the performance improvements in these configurations are lower than those observed in the configuration with 64 SMs per chip. We infer that the increased number of SMs leads to more intense resource contention, which raises the overhead of exploiting inter-chip locality. It is observed that the configuration with 96 SMs achieves greater performance improvement than that with 80 SMs. This may be due to the increased remote data cache capacity provided by the additional SMs, which allows for more exploitation of inter-chip locality. Overall, TLS-Cache outperforms the other three architectures across all SM configurations, demonstrating its robustness under varying SM configurations.

Effect of Chip Count. We compare the performance of all four architectures under configurations with different numbers of chips. We present the performance improvements of each architecture in various configurations (chip-4, chip-6, and chip-8) in Fig. 12(b). Under the chip-8 and chip-6 configurations, inter-chip communication is implemented using a classic X-Y routing scheme. As shown in Fig. 12(b), TLS-Cache achieves significant performance gains compared with Baseline as the number of chips increases in multi-chip GPUs. In the 8-chip configuration, TLS-Cache improves performance by 42.0% over Baseline, which is higher than the 30.2% improvement observed in the 4-chip configuration. L1.5 also shows increasing performance improvements as the number of chips grows, by exploiting intra-chip locality. In contrast, Shared L1 provides little performance gain due to its insufficient utilization of intra-chip locality. These results show that as the number of chips increases, the latency of remote memory access becomes a critical performance bottleneck in multi-chip GPUs. Therefore, TLS-Cache shows considerable potential for performance improvement in future multi-chip GPUs with a large number of chips.

Effect of L1 Cache Size. We evaluate the impact of different L1 cache sizes on the performance of all four architectures, as shown in Fig. 12(c). The figure shows the performance of each architecture normalized to Baseline for L1 cache sizes of 64K, 128K, and 256K per SM. We observe that applications can be categorized into those sensitive to L1 cache size and those not. For L1 cache-sensitive applications, such as *CFD* and *SRAD*, the performance improvement of TLS-Cache over Baseline increases with larger L1 cache sizes. This is because a larger L1 cache results in a larger remote-shared L1 cache, which reduces remote memory accesses and thus provides greater performance gains. On the other hand, applications like *SORT* and *FFT*, which have very low L1 cache hit rates, are less sensitive to L1 cache size. For these applications, a small remote-shared L1 cache is sufficient to store remote memory data, so the performance gains from increasing the L1 cache size are minimal. Overall, TLS-Cache delivers performance improvements across various L1 cache sizes.

Effect of Page Allocation Policy. Different page allocation policies result in varying amounts of inter-chip memory accesses. We evaluate the performance improvement of all four architectures under round-robin and first-touch page allocation. As shown in Fig. 12(d), TLS-Cache has considerable performance improvements under both page allocation policies. This is because, under both policies, performance bottlenecks caused by remote data accesses still exist [14,25]. TLS-Cache exploits intra-chip locality to reduce remote memory accesses, thereby improving performance. Although L1.5 and Shared L1 also deliver moderate performance gains, TLS-Cache shows better performance than the other three architectures under both page allocation policies.

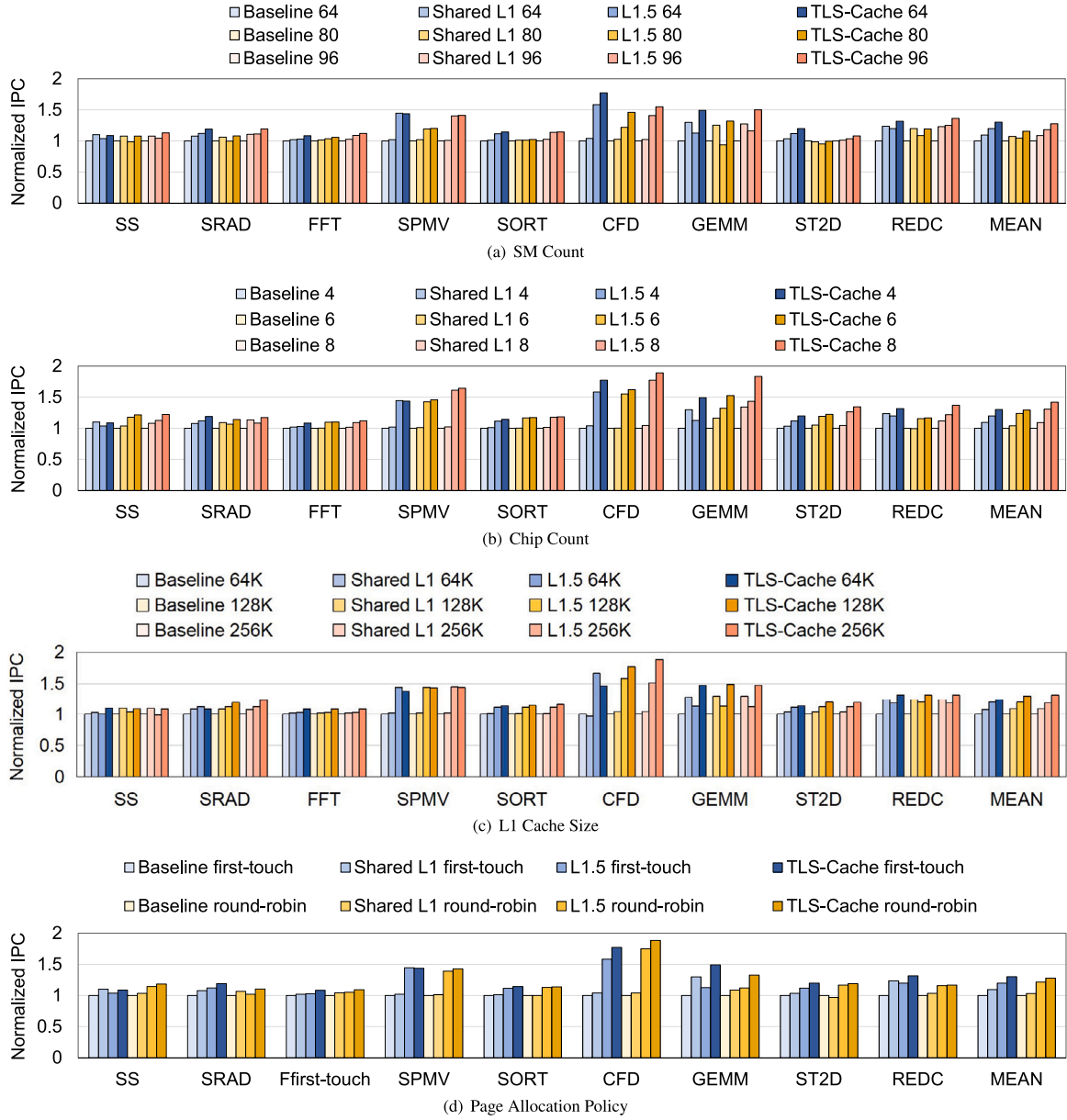


Fig. 12. Performance variation of TLS-Cache across different configurations (normalized to Baseline).

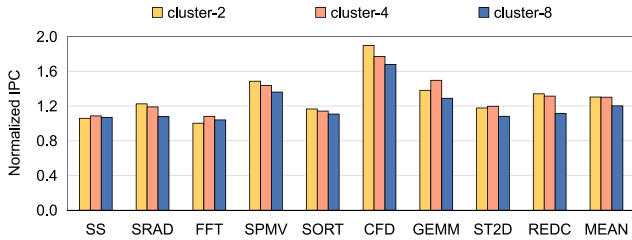


Fig. 13. Performance variation of TLS-Cache with different cluster sizes.

Effect of Cluster Size. In TLS-Cache, we divide the SMs within the chip into multiple clusters and allocate the L1 caches of the SMs in each cluster as cluster-shared L1 caches and remote-shared L1 caches. In our experiments, each cluster is configured with one remote-shared L1 cache, while the remaining L1 caches serve as cluster-shared caches. As Baseline and L1.5 do not adopt cluster-based designs, and Shared L1 lacks remote-shared L1 caches within clusters, the impact of cluster

size is evaluated only for TLS-Cache. The results are presented in Fig. 13. As shown in the figure, the average performance improvement of cluster-2 and cluster-4 over Baseline is comparable, which are 30.4% and 30.2%, respectively. For applications such as *CFD*, *SRAD*, and *SPMV*, the cluster-2 configuration performs better, while the cluster-4 configuration shows better performance for applications like *FFT* and *GEMM*. When the cluster size increases to 8, the performance improvement of TLS-Cache drops to 20.2%. This is primarily because the capacity of the remote-shared L1 cache decreases, which is only one-eighth of the total L1 cache capacity, resulting in a decrease in overall performance. Therefore, TLS-Cache requires careful tuning of cluster size or the number of remote-shared L1 caches to maintain high performance. Based on our experimental results, setting the cluster size to 4, with three cluster-shared L1 caches and one remote-shared L1 cache per cluster, proves to be a recommended configuration.

4.6. Overhead

Performance Overhead. In TLS-Cache, requests from each SM need to access the L1 caches of other SMs. This process introduces additional

data transfer delays as well as latency due to resource contention. However, as discussed in previous studies [10,11,27,34], the performance overhead introduced by cache sharing is negligible compared with the performance gains for applications with high intra-chip locality. Furthermore, the two-level crossbar introduced in TLS-Cache results in higher latency compared with the crossbar in traditional GPUs. In our experiments, we simulate all these additional performance overheads in detail, and the results show that TLS-Cache still achieves performance gains.

Hardware Overhead. The main hardware overhead of the special design in TLS-Cache comes from the two-level crossbar used to enable data exchange between SMs. As shown in Fig. 3, assuming each GPU chip has 64 SMs and each cluster has 4 SMs, there are a total of 16 clusters per chip. Therefore, we need 16 4x4 intra-cluster crossbars and one 16x16 intra-chip crossbar. In addition, in the request router shown in Fig. 5, the additional designs mainly include Tag Selectors and Cache Mapping Units. These are hardware components with very simple hardware logic and minimal area.

The additional cost of TLS-Cache has been synthesized using Synopsys tools in a 28 nm process. The synthesized report indicates that our design results in an additional 3.48 mm² of area and 2.75 W of power consumption in a chip. The additional area and power consumption costs for TLS-Cache account for approximately 0.57% and 0.92%, respectively, of the typical GPU chip's area and power consumption [28,35,36].

5. Related work

5.1. Locality exploitation in multi-chip gpus

Previous studies have improved the performance of multi-chip GPUs by exploiting data locality. Modifying the cache hierarchy to reduce inter-chip memory accesses is a promising approach. The remote data cache architecture adds a new cache level between the L1 and last-level cache, used exclusively for caching data from remote memory [1,3,6]. Arunkumar et al. [1] propose MCM-GPU and present using remote-only L1.5 cache to reduce remote memory accesses. Belayneh et al. propose DualOpt to reduce the remote memory access latency by fine-granularity transfer and remote data caching [6]. Young et al. propose Caching Remote Data in Video Memory (CARVE), which stores remote data in a portion of the GPU on-package local memory [3]. Another new cache hierarchy modifies the traditional memory-side last-level cache in the GPU to an SM-side last-level cache. The SM-side last-level cache acts as an extended L1 cache within the chip, caching data from the entire address space [4,7,25]. Milic et al. [4] propose NUMA-Aware GPUs that dynamically adapt the caching policy to minimize NUMA effects by the SM-side last-level cache. Zhang et al. characterize data sharing in Multi-GPU systems and propose Sharing-Aware Caching (SAC), which adopts either a memory-side or SM-side last-level cache organization based on a simple analytical model [7,25]. Other studies improve the GPU performance through methods such as data placement [37], data migration [14,38,39], data coalescing [24,33,40], memory-centric architecture [41], and page table walk optimization [42,43].

Compared with other studies (e.g., SAC [7], NUBA [14]), TLS-Cache employs a different approach by exploiting intra-chip locality at the L1 cache level. Since our design operates at the L1 cache, it is complementary to approaches that target the last-level cache, data placement, data migration, etc., enabling further performance improvements.

5.2. GPU shared L1 cache

The shared L1 cache architecture connects the private L1 caches of multiple SMs through a NoC to achieve data sharing across SMs. This design efficiently utilizes the locality between different SMs within the GPU. Dubliss et al. [26] propose an L1 cooperative cache network that

enables data sharing among L1 caches in the GPU via a ring network. Ibrahim et al. [27] use a mesh network to connect the L1 caches and introduce a prediction mechanism to determine whether to share the caches. Decoupled-sharing cache [10,34] maps the same address requests to the same cache according to the address mapping rules, thereby reducing data duplication between L1 caches. Xu et al. [11] propose ATA-Cache, which can reduce the additional resource contention caused by shared L1 cache. Wang et al. [12] propose a shared L1 cache architecture that leverages the locality between L1 cache groups. Falahati et al. propose Cross-Core Data Sharing (CCDS), using a predictor to estimate whether the required data exists in another SM [13].

It is worth noting that NVIDIA has proposed distributed shared memory in H100 [44], but this design only allows the shared memory to be shared between SMs, not the L1 cache. This indicates that our shared L1 cache architecture has potential application prospects in industrial products because both shared L1 cache and distributed shared memory use similar interconnection schemes between SMs.

6. Conclusion and future work

In this article, we discuss how to exploit intra-chip locality to reduce remote memory accesses and mitigate the performance bottlenecks of multi-chip GPUs. We observe that the shared L1 cache provides an opportunity to exploit intra-chip locality by designing a unified architecture for the L1 and remote data cache. We propose a fine-grained two-level shared L1 cache architecture, where SMs are organized into clusters, and the L1 caches within each cluster are organized into cluster-shared L1 caches and remote-shared L1 caches, capturing both intra-cluster and inter-cluster locality. Compared with the remote data cache architecture, TLS-Cache exploits intra-chip locality with lower overhead, resulting in more substantial performance improvements. Experimental results show that TLS-Cache improves performance by 30.2% over the baseline multi-chip GPU and by 8.8% over the remote data cache architecture. Our work demonstrates that rethinking the L1 cache hierarchy in multi-chip GPUs can significantly reduce cross-chip traffic and enhance system performance.

In the future, integrating TLS-Cache with GPU workload locality analysis [37,45] could enable dynamic, locality-aware cache configuration for further performance optimization. Additionally, we plan to combine TLS-Cache with other locality capture methods [7,14,24], which could further enhance performance.

CRedit authorship contribution statement

Xiangrong Xu: Writing – review & editing, Writing – original draft, Validation, Methodology, Data curation. **Liang Wang:** Writing – review & editing, Writing – original draft, Validation, Supervision, Methodology. **Limin Xiao:** Writing – review & editing, Writing – original draft, Validation, Supervision. **Lei Liu:** Writing – review & editing, Writing – original draft, Validation, Supervision. **Zihao Zhou:** Writing – original draft, Methodology. **Yuanqiu Lv:** Writing – original draft, Validation. **Li Ruan:** Writing – review & editing, Writing – original draft. **Xilong Xie:** Writing – review & editing, Writing – original draft. **Meng Han:** Writing – review & editing, Writing – original draft. **Xiaojuan Liao:** Writing – review & editing, Writing – original draft, Validation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

- [1] A. Arunkumar, E. Bolotin, B.Y. Cho, U. Milic, E. Ebrahimi, O. Villa, A. Jaleel, C. Wu, D.W. Nellans, MCM-GPU: multi-chip-module GPUs for continued performance scalability, in: Proceedings of the 44th Annual International Symposium on Computer Architecture, ISCA 2017, Toronto, on, Canada, June 24–28, 2017, ACM, 2017, pp. 320–332, <http://dx.doi.org/10.1145/3079856.3080231>.
- [2] Y. Feng, S. Na, H. Kim, H. Jeon, Barre chord: Efficient virtual memory translation for multi-chip-module GPUs, in: Proceedings of the 51st Annual International Symposium on Computer Architecture, ISCA 2024, Buenos Aires, Argentina, June 29 - July 3, 2024, IEEE, 2024, pp. 834–847, <http://dx.doi.org/10.1109/ISCA59077.2024.00065>.
- [3] V. Young, A. Jaleel, E. Bolotin, E. Ebrahimi, D.W. Nellans, O. Villa, Combining HW/SW mechanisms to improve NUMA performance of multi-gpu systems, in: Proceedings of the 51st Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2018, Fukuoka, Japan, October 20–24, 2018, IEEE Computer Society, 2018, pp. 339–351, <http://dx.doi.org/10.1109/MICRO.2018.00035>.
- [4] U. Milic, O. Villa, E. Bolotin, A. Arunkumar, E. Ebrahimi, A. Jaleel, A. Ramírez, D.W. Nellans, Beyond the socket: NUMA-aware GPUs, in: Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2017, Cambridge, MA, USA, October 14–18, 2017, ACM, 2017, pp. 123–135, <http://dx.doi.org/10.1145/3123939.3124534>.
- [5] G. Ko, J. Lee, H. Kal, H. Lee, W. Ro, REC: enhancing fine-grained cache coherence protocol in multi-GPU systems, J. Syst. Arch. 160 (2025) 103339, <http://dx.doi.org/10.1016/J.SYSARC.2025.103339>.
- [6] L. Belayneh, H. Ye, K. Chen, D.T. Blaauw, T.N. Mudge, R.G. Dreslinski, N. Talati, Locality-aware optimizations for improving remote memory latency in multi-GPU systems, in: Proceedings of the 31st International Conference on Parallel Architectures and Compilation Techniques, PACT 2022, Chicago, Illinois, October 8–12, 2022, ACM, 2022, pp. 304–316, <http://dx.doi.org/10.1145/3559009.3569649>.
- [7] S. Zhang, M. Naderan-Tahan, M. Jahre, L. Eeckhout, SAC: sharing-aware caching in multi-chip GPUs, in: Proceedings of the 50th Annual International Symposium on Computer Architecture, ISCA 2023, Orlando, FL, USA, June 17–21, 2023, ACM, 2023, pp. 43:1–43:13, <http://dx.doi.org/10.1145/3579371.3589078>.
- [8] X. Zhao, A. Adileh, Z. Yu, Z. Wang, A. Jaleel, L. Eeckhout, Adaptive memory-side last-level GPU caching, in: Proceedings of the 46th Annual International Symposium on Computer Architecture, ISCA 2019, Phoenix, AZ, USA, June 22–26, 2019, ACM, 2019, pp. 411–423, <http://dx.doi.org/10.1145/3307650.3322235>.
- [9] X. Zhao, Y. Liu, A. Adileh, L. Eeckhout, LA-LLC: inter-core locality-aware last-level cache to exploit many-to-many traffic in GPGPUs, IEEE Comput. Arch. Lett. 16 (1) (2017) 42–45, <http://dx.doi.org/10.1109/LCA.2016.2611663>.
- [10] M.A. Ibrahim, O. Kayiran, Y. Eckert, G.H. Loh, A. Jog, Analyzing and leveraging decoupled L1 caches in GPUs, in: 2021 IEEE International Symposium on High-Performance Computer Architecture, HPCA 2021, Seoul, South Korea, February 27 - March 3, 2021, IEEE, 2021, pp. 467–478, <http://dx.doi.org/10.1109/HPCA51647.2021.00047>.
- [11] X. Xu, L. Wang, L. Xiao, L. Liu, Y. Lv, X. Xie, M. Han, H. Liu, ATA-cache: Contention mitigation for GPU shared L1 cache with aggregated tag array, IEEE Trans. Comput. Aided Des. Integr. Circuits Syst. 43 (5) (2024) 1429–1441, <http://dx.doi.org/10.1109/TCAD.2023.3337192>.
- [12] G. Wang, Y. Du, W. Huang, GPU performance optimization via intergroup cache cooperation, IEEE Trans. Comput. Aided Des. Integr. Circuits Syst. 43 (11) (2024) 4142–4153, <http://dx.doi.org/10.1109/TCAD.2024.3443707>.
- [13] H. Falahati, M. Sadrosadati, Q. Xu, J. Gómez-Luna, B.S. Latibari, H. Jeon, S. Hessabi, H. Sarbazi-Azad, O. Mutlu, M. Annavaram, M. Pedram, Cross-core data sharing for energy-efficient GPUs, ACM Trans. Arch. Code Optim. 21 (3) (2024) 42:1–42:32, <http://dx.doi.org/10.1145/3653019>.
- [14] X. Zhao, M. Jahre, Y. Tang, G. Zhang, L. Eeckhout, NUBA: non-uniform bandwidth GPUs, in: Proceedings of the 28th International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2023, Vancouver, BC, Canada, March 25–29, 2023, ACM, 2023, pp. 544–559, <http://dx.doi.org/10.1145/3575693.3575745>.
- [15] T. Wu, J. He, Y. Qian, W. Liu, Optimizing the performance of in-memory file system by thread scheduling and file migration under NUMA multiprocessor systems, J. Syst. Arch. 159 (2025) 103344, <http://dx.doi.org/10.1016/J.SYSARC.2025.103344>.
- [16] A. Bakhoda, G.L. Yuan, W.W.L. Fung, H. Wong, T.M. Aamodt, Analyzing CUDA workloads using a detailed GPU simulator, in: IEEE International Symposium on Performance Analysis of Systems and Software, ISPASS 2009, April 26–28, 2009, Boston, Massachusetts, USA, Proceedings, IEEE Computer Society, 2009, pp. 163–174, <http://dx.doi.org/10.1109/ISPASS.2009.4919648>.
- [17] M. Khairy, Z. Shen, T.M. Aamodt, T.G. Rogers, Accel-sim: An extensible simulation framework for validated GPU modeling, in: Proceedings of the 47th Annual International Symposium on Computer Architecture, ISCA 2020, Virtual Event / Valencia, Spain, May 30 - June 3, 2020, IEEE, 2020, pp. 473–486, <http://dx.doi.org/10.1109/ISCA45697.2020.00047>.
- [18] L. Liu, S. Yang, L. Peng, X. Li, Hierarchical hybrid memory management in OS for tiered memory systems, IEEE Trans. Parallel Distrib. Syst. 30 (10) (2019) 2223–2236, <http://dx.doi.org/10.1109/TPDS.2019.2908175>.
- [19] Q. Wang, Z. Peng, B. Ren, J. Chen, R.G. Edwards, MemHC: An optimized GPU memory management framework for accelerating many-body correlation, ACM Trans. Arch. Code Optim. 19 (2) (2022) 24:1–24:26, <http://dx.doi.org/10.1145/3506705>.
- [20] R. Ausavarungnirun, V. Miller, J. Landgraf, S. Ghose, J. Gandhi, A. Jog, C.J. Rossbach, O. Mutlu, MASK: redesigning the GPU memory hierarchy to support multi-application concurrency, in: Proceedings of the 23rd International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS 2018, Williamsburg, VA, USA, March 24–28, 2018, ACM, 2018, pp. 503–518, <http://dx.doi.org/10.1145/3173162.3173169>.
- [21] R. Ausavarungnirun, J. Landgraf, V. Miller, S. Ghose, J. Gandhi, C.J. Rossbach, O. Mutlu, Mosaic: a GPU memory manager with application-transparent support for multiple page sizes, in: Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2017, Cambridge, MA, USA, October 14–18, 2017, ACM, 2017, pp. 136–150, <http://dx.doi.org/10.1145/3123939.3123975>.
- [22] X. Peng, X. Shi, H. Dai, H. Jin, W. Ma, Q. Xiong, F. Yang, X. Qian, Capuchin: Tensor-based GPU memory management for deep learning, in: Proceedings of the 25th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Lausanne, Switzerland, March 16–20, 2020, ACM, 2020, pp. 891–905, <http://dx.doi.org/10.1145/3373376.3378505>.
- [23] P. Dalmia, R. Kumar, M. Sinclair, CPElide: Efficient multi-chiplet GPU implicit synchronization, in: Proceedings of the 57th IEEE/ACM International Symposium on Microarchitecture, MICRO 2024, Austin, TX, USA, November 2–6, 2024, IEEE, 2024, pp. 700–717, <http://dx.doi.org/10.1109/MICRO61859.2024.00058>.
- [24] X. Zhang, G. Zhang, L. Wang, S. Zhang, X. Zhao, Adcoalescer: An adaptive coalescer to reduce the inter-module traffic in MCM-gpus, in: Proceedings of the 53rd International Conference on Parallel Processing, ICPP 2024, Gotland, Sweden, August 12–15, 2024, ACM, 2024, pp. 1001–1011, <http://dx.doi.org/10.1145/3673038.3673075>.
- [25] S. Zhang, M. Naderan-Tahan, M. Jahre, L. Eeckhout, Characterizing multi-chip GPU data sharing, ACM Trans. Arch. Code Optim. 20 (4) (2023) 56:1–56:24, <http://dx.doi.org/10.1145/3629521>.
- [26] S. Dublisch, V. Nagarajan, N.P. Topham, Cooperative caching for GPUs, ACM Trans. Arch. Code Optim. 13 (4) (2016) 39:1–39:25, <http://dx.doi.org/10.1145/3001589>.
- [27] M.A. Ibrahim, H. Liu, O. Kayiran, A. Jog, Analyzing and leveraging remote-core bandwidth for enhanced performance in GPUs, in: Proceedings of the 28th International Conference on Parallel Architectures and Compilation Techniques, PACT 2019, Seattle, WA, USA, September 23–26, 2019, IEEE, 2019, pp. 258–271, <http://dx.doi.org/10.1109/PACT.2019.00028>.
- [28] NVIDIA, NVIDIA TESLA V100 GPU ARCHITECTURE, Technical Report, NVIDIA, 2017, URL <https://images.nvidia.com/content/volta-architecture/pdf/volta-architecture-whitepaper.pdf>.
- [29] B. He, W. Fang, Q. Luo, N.K. Govindaraju, T. Wang, Mars: a MapReduce framework on graphics processors, in: Proceedings of the 17th International Conference on Parallel Architectures and Compilation Techniques, PACT 2008, Toronto, Ontario, Canada, October 25–29, 2008, ACM, 2008, pp. 260–269, <http://dx.doi.org/10.1145/1454115.1454152>.
- [30] S. Che, M. Boyer, J. Meng, D. Tarjan, J.W. Sheaffer, S. Lee, K. Skadron, Rodinia: A benchmark suite for heterogeneous computing, in: Proceedings of the 2009 IEEE International Symposium on Workload Characterization, IISWC 2009, October 4–6, 2009, Austin, TX, USA, IEEE Computer Society, 2009, pp. 44–54, <http://dx.doi.org/10.1109/IISWC.2009.5306797>.
- [31] A. Danalis, G. Marin, C. McCurdy, J.S. Meredith, P.C. Roth, K. Spafford, V. Tipparaju, J.S. Vetter, The scalable heterogeneous computing (SHOC) benchmark suite, in: Proceedings of the 3rd Workshop on General Purpose Processing on Graphics Processing Units, GPGPU 2010, Pittsburgh, Pennsylvania, USA, March 14, 2010, in: ACM International Conference Proceeding Series, 425, ACM, 2010, pp. 63–74, <http://dx.doi.org/10.1145/1735688.1735702>.
- [32] J.A. Stratton, C. Rodrigues, I.-J. Sung, N. Obeid, L.-W. Chang, N. Anssari, G.D. Liu, W.-m.W. Hwu, Parboil: A revised benchmark suite for scientific and commercial throughput computing, Technical Report, University of Illinois at Urbana-Champaign, 2012.
- [33] X. Zhao, G. Zhang, L. Wang, S. Zhang, H. Dai, NearFetch: Saving inter-module bandwidth in many-chip-module GPUs, in: 2025 IEEE International Symposium on High Performance Computer Architecture, HPCA 2025, Las Vegas, NV, USA, March 1–5, 2025, IEEE, 2025, pp. 1693–1706, <http://dx.doi.org/10.1109/HPCA61900.2025.00125>.
- [34] M.A. Ibrahim, O. Kayiran, Y. Eckert, G.H. Loh, A. Jog, Analyzing and leveraging shared L1 caches in GPUs, in: Proceedings of the 29th International Conference on Parallel Architectures and Compilation Techniques, PACT 2020, Virtual Event, GA, USA, October 3–7, 2020, ACM, 2020, pp. 161–173, <http://dx.doi.org/10.1145/3410463.3414623>.
- [35] J. Choquette, W. Gandhi, NVIDIA A100 GPU: performance & innovation for GPU computing, in: 2020 IEEE Hot Chips 32 Symposium, HCS 2020, Palo Alto, CA, USA, August 16–18, 2020, IEEE, 2020, pp. 1–43, <http://dx.doi.org/10.1109/HCS49909.2020.9220622>.

- [36] J. Choquette, Nvidia hopper GPU: scaling performance, in: 2022 IEEE Hot Chips 34 Symposium, HCS 2022, Cupertino, CA, USA, August 21-23, 2022, IEEE, 2022, pp. 1–46, <http://dx.doi.org/10.1109/HCS55958.2022.9895592>.
- [37] M. Khairy, V. Nikiforov, D.W. Nellans, T.G. Rogers, Locality-centric data and threadblock management for massive GPUs, in: Proceedings of the 53rd Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2020, Athens, Greece, October 17-21, 2020, IEEE, 2020, pp. 1022–1036, <http://dx.doi.org/10.1109/MICRO50266.2020.00086>.
- [38] T. Baruah, Y. Sun, A.T. Dinçer, S.A. Mojumder, J.L. Abellán, Y. Ukidave, A. Joshi, N. Rubin, J. Kim, D.R. Kaeli, Griffin: Hardware-software support for efficient page migration in multi-GPU systems, in: 2020 IEEE International Symposium on High-Performance Computer Architecture, HPCA 2020, San Diego, CA, USA, February 22-26, 2020, IEEE, 2020, pp. 596–609, <http://dx.doi.org/10.1109/HPCA47549.2020.00055>.
- [39] H. Muthukrishnan, D. Lustig, D.W. Nellans, T.F. Wenisch, GPS: a global publish-subscribe model for multi-GPU memory management, in: Proceedings of the 54th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2021, Virtual Event, Greece, October 18-22, 2021, ACM, 2021, pp. 46–58, <http://dx.doi.org/10.1145/3466752.3480088>.
- [40] L. Wang, X. Zhao, D.R. Kaeli, Z. Wang, L. Eeckhout, Intra-cluster coalescing and distributed-block scheduling to reduce GPU NoC pressure, IEEE Trans. Comput. 68 (7) (2019) 1064–1076, <http://dx.doi.org/10.1109/TC.2019.2895036>.
- [41] H. SeyyedAghaei, M. Naderan-Tahan, M. Jahre, L. Eeckhout, Memory-Centric MCM-GPU Architecture, IEEE Comput. Arch. Lett. 24 (1) (2025) 101–104, <http://dx.doi.org/10.1109/LCA.2025.3553766>.
- [42] B. Pratheek, N. Jawalkar, A. Basu, Designing virtual memory system of MCM GPUs, in: Proceedings of the 55th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2022, Chicago, IL, USA, October 1-5, 2022, IEEE, 2022, pp. 404–422, <http://dx.doi.org/10.1109/MICRO56248.2022.00036>.
- [43] B. Li, J. Yin, A. Holey, Y. Zhang, J. Yang, X. Tang, Trans-FW: Short circuiting page table walk in multi-GPU systems via remote forwarding, in: 2023 IEEE International Symposium on High-Performance Computer Architecture, HPCA 2023, Montreal, QC, Canada, February 25-March 1, 2023, IEEE, 2023, pp. 456–470, <http://dx.doi.org/10.1109/HPCA56546.2023.10071054>.
- [44] NVIDIA, NVIDIA H100 Tensor Core GPU Architecture, 2023, URL <https://resources.nvidia.com/en-us-tensor-core>.
- [45] N. Vijaykumar, E. Ebrahimi, K. Hsieh, P.B. Gibbons, O. Mutlu, The locality descriptor: A holistic cross-layer abstraction to express data locality in GPUs, in: Proceedings of the 45th Annual International Symposium on Computer Architecture, ISCA 2018, Los Angeles, CA, USA, June 1-6, 2018, IEEE Computer Society, 2018, pp. 829–842, <http://dx.doi.org/10.1109/ISCA.2018.00074>.



Xiangrong Xu received the B.S. degree in Computer Science from Beihang University, Beijing, China, in 2020. He is currently working toward the Ph.D. degree in Computer Architecture with the School of Computer Science and Engineering, Beihang University, Beijing, China. His research interests include computer architecture and GPU. He has a good understanding of GPU cache architecture and GPU task scheduling. He is currently designing and coding a GPU performance simulator.



Liang Wang received the BEng and MSc degree in electronics engineering from Harbin Institute of Technology, China, in 2011 and 2013 respectively, and the Ph.D. degree in Computer Science and Engineering from The Chinese University of Hong Kong in 2017. He is currently an assistant professor with the School of Computer Science and Engineering, Beihang University, China. He was a postdoctoral research fellow in Institute of Microelectronics, Tsinghua University from 2018 to 2020. His research interests include computer architecture, heterogeneous computing systems, etc.



Limin Xiao, Ph.D., Professor of Beihang University, CCF senior member and IEEE member. His research interests include computer architecture and system software, high performance computer and server system, system virtualization and cloud computing, big data storage and distributed file system, computer system security. He authored more than 240 papers on international journals and conferences.



Lei Liu is a Full Professor of Computer Science at Beihang University. He has led research projects that have advanced the cloud native technologies, involving memory systems, OS, AI for Systems, performance isolation, and profiling. He has published research articles regarding the architecture and operating system in venues that include ISCA, HPCA, FAST, PACT, IEEE TC, TPDS, ACM TACO, etc.



Zihao Zhou is currently an undergraduate student pursuing a B.S. degree in Computer Science at Beihang University, Beijing, China. He has a strong passion for computer architecture and a dedicated focus on advancing research in GPU (Graphics Processing Unit) technologies. His research interests include GPU architecture design, parallel computing, and performance optimization. He is actively exploring GPU memory hierarchy optimization and task scheduling strategies, with practical experience in developing GPU-related simulation tools.



Yuanqiu Lv received the B.S. degree in Computer Science from Beihang University, Beijing, China, in 2024. She has a strong passion for computer architecture and a keen interest in advancing knowledge in the field of GPU (Graphics Processing Unit). Her research interests include computer architecture, operating system and GPU performance modeling.



Li Ruan received the PhD degree in computer science from Institute of Software, Chinese Academy of Sciences in 2009. She is currently an associate professor at School of Computer Science and Engineering, Beihang University, China. Her research interests include intelligent data analysis and security, autonomous driving, parallel and distributing computing, cloud computing, big data analysis, and visualization.



Xilong Xie received the B.S. degree in Computer Science from the China University of Petroleum, Qingdao, China in 2022. He is currently working toward the Ph.D. degree in Computer Architecture with the School of Computer Science and Engineering, Beihang University, Beijing, China. His research interests include computer architecture and RISC-V CPU.



Meng Han received the B.S. degree in Computer Science from Beijing University of Posts and Telecommunications, Beijing, China, in 2019. He completed his Ph.D. in Computer Architecture at Beihang University, Beijing, China, in 2025. His research interests include computer architecture, deep learning accelerators, and high-performance computing. He is currently a postdoctoral researcher at Tsinghua University, Beijing, China.



Xiaojian Liao is an associate professor in the School of Computer Science and Engineering, Beihang University. He received the PhD degree from Tsinghua University. His research interests are primarily in storage systems and AI systems, focusing on enhancing their performance and reliability. His work has been published in top-tier conferences and journals on computer systems, including OSDI, SOSP, FAST, USENIX ATC, EuroSys, ASPLOS, HPCA and ACM TOS.